

- Practice Sheet Exercise 1: Customer Information Procedure

Create a stored procedure that:

- a) Takes a customer\_id as input
- b) Returns the customer's full name, total number of orders, and total amount spent
- c) Uses variables to store intermediate calculations

```
CREATE PROCEDURE GetCustomerInfo
    @CustomerId INT
AS
BEGIN
    DECLARE @FullName NVARCHAR(200)
    DECLARE @TotalOrders INT
    DECLARE @TotalSpent DECIMAL(10,2)
    -- return full name
    SELECT
        @FullName = first_name + ' ' + last_name
    FROM sales.customers
    WHERE customer_id = @CustomerId
    -- Count orders
    SELECT
        @TotalOrders = COUNT(*)
    FROM sales.orders
    WHERE customer_id = @CustomerId
    -- Sum total spent after discount
    SELECT
        @TotalSpent = ISNULL(SUM(OI.quantity * OI.list_price * (1 - OI.discount)),0)
    FROM sales.orders O
    JOIN sales.order_items OI
        ON O.order_id = OI.order_id
    WHERE O.customer_id = @CustomerId
    -- Return result
    SELECT
        @FullName AS FullName,
        @TotalOrders AS TotalOrders,
        @TotalSpent AS TotalSpent
END
```

The Result :

The screenshot shows a SQL query execution window. At the top, two SQL statements are entered: `exec GetCustomerInfo 1` and `exec GetCustomerInfo 100`. Below the statements, a zoom level of 102% is indicated. The 'Results' tab is active, displaying two tables of data. The first table, corresponding to the first statement, has columns 'FullName', 'TotalOrders', and 'TotalSpent', with one row for 'Debra Burks'. The second table, corresponding to the second statement, has the same columns and one row for 'Kellie Franco'.

```
exec GetCustomerInfo 1
exec GetCustomerInfo 100
```

102 %

Results Messages

|   | FullName    | TotalOrders | TotalSpent |
|---|-------------|-------------|------------|
| 1 | Debra Burks | 3           | 27888.18   |

  

|   | FullName      | TotalOrders | TotalSpent |
|---|---------------|-------------|------------|
| 1 | Kellie Franco | 1           | 5399.98    |

## Exercise 2: Stock Checker Procedure

Create a procedure that:

- **a)** Takes store\_id and product\_id as parameters
- **b)** Checks the stock quantity from the stocks table

```

-- question 2

CREATE PROCEDURE CheckProductatStock
    @StoreId INT,
    @ProductId INT
AS
BEGIN
    DECLARE @Quantity INT
    DECLARE @Message NVARCHAR(100)

    SELECT @Quantity = quantity
    FROM production.stocks
    WHERE store_id = @StoreId
    AND product_id = @ProductId

    IF @Quantity IS NULL
        SET @Message = 'Product not carried'

    ELSE IF @Quantity = 0
        SET @Message = 'Out of Stock'

    ELSE IF @Quantity BETWEEN 1 AND 5
        SET @Message = 'Low Stock (' + CAST(@Quantity AS NVARCHAR) + ' units)'

    ELSE
        SET @Message = 'In Stock (' + CAST(@Quantity AS NVARCHAR) + ' units)'

    SELECT @Message AS StockStatus
END

```

The Result :

|      |                     |       |
|------|---------------------|-------|
| EXEC | CheckProductatStock | 1,1   |
| EXEC | CheckProductatStock | 1,300 |
| EXEC | CheckProductatStock | 50,1  |
| EXEC | CheckProductatStock | 1,6   |
| EXEC | CheckProductatStock | 1,2   |

  

| Results |             | Messages            |
|---------|-------------|---------------------|
| 1       | StockStatus | In Stock (27 units) |
| 1       | StockStatus | In Stock (7 units)  |
| 1       | StockStatus | Product not carried |
| 1       | StockStatus | Out of Stock        |
| 1       | StockStatus | Low Stock (5 units) |

### Exercise 3: Price Update Procedure

Create a procedure that:

- a) Takes brand\_id and percentage\_increase as parameters
- b) Uses a WHILE loop to update prices of all products for that brand
- c) Applies the percentage increase to each product's list\_price
- d) Returns the number of products updated

```
CREATE PROCEDURE UpdateBrandPrices
    @BrandId INT,
    @PercentageIncrease DECIMAL(5,2)
AS
BEGIN
    DECLARE @ProductId INT
    DECLARE @CurrentPrice DECIMAL(10,2)
    DECLARE @NewPrice DECIMAL(10,2)
    DECLARE @Counter INT = 0

    DECLARE @Products TABLE (
        ProductId INT,
        ListPrice DECIMAL(10,2))

    INSERT INTO @Products
    SELECT product_id, list_price
    FROM production.products
    WHERE brand_id = @BrandId

    DECLARE @Index INT = 1
    DECLARE @Total INT

    SELECT @Total = COUNT(*) FROM @Products

    WHILE @Index <= @Total
    BEGIN
        SELECT
            @ProductId = ProductId,
            @CurrentPrice = ListPrice
        FROM (
            SELECT
                ProductId,
                ListPrice,
                ROW_NUMBER() OVER (ORDER BY ProductId) AS rn
            FROM @Products ) t
        WHERE rn = @Index
        SET @NewPrice = @CurrentPrice * (1 + @PercentageIncrease / 100.0)
        UPDATE production.products
        SET list_price = @NewPrice
        WHERE product_id = @ProductId
        SET @Counter = @Counter + 1
        SET @Index = @Index + 1
    END
```

The Result :

```
EXEC UpdateBrandPrices 1, 10
```

```
EXEC UpdateBrandPrices 2, 5
```

```
SELECT TOP 4 store_id, product_id
```

5 %

Results Messages

UpdatedProducts

118

UpdatedProducts

10